CART CRASH MOUNTAIN

Go downhill. Get paid. Bolt on something ridiculous.

A Roblox physics arcade game where players send shopping carts down a giant mountainside, earn cash from distance and stunts, and return with increasingly outrageous machines.

The promise

A new player should understand the game by watching one cart launch. The first run starts almost immediately. Crashing is funny, progress is visible, and the next upgrade always suggests something new to try.

Your requested feature

After earning enough in-game money, players unlock a workshop that lets them place items on their cart and attach them with **Z** or a large **Weld** button. Turbines, boosters, suspension and other parts change how the cart behaves.

Decision	Plan
Genre	Casual downhill physics / upgrade arcade
Audience	Broad Roblox audience; simple controls and readable rewards
Typical run	45-90 seconds; rapid return after a crash
Play modes	Solo driving in a shared server; friends race together
Main progression	Stock cart > basic upgrades > workshop > custom machines
Document status	Build proposal. All prices, physics values and targets require playtesting.

Working title; name availability has not been checked. This is a design proposal, not a validated revenue forecast or a claim of originality.

The game in one session

The repeatable loop

Launch > steer and survive > collect cash and perform stunts > crash or finish > receive payout > upgrade > launch again.

The first five minutes

Time	Player experience
0:00-0:20	Spawn beside a stock cart with a bright Ride prompt. Sit and launch with one button. No shop tutorial first.
0:20-1:10	Slide down an easy opening slope. Collect a coin line, hit a small ramp and learn steering.
1:10-1:25	Crash or reach a checkpoint. See a short distance / stunts / cash breakdown and an immediate Again button.
1:25-2:30	Buy an affordable first upgrade. A larger wheel or colored booster makes the change visible.
2:30-5:00	Attempt a better route, see friends launch, and get a clear preview of the workshop unlock.

Design pillars

• **Understand instantly:** downhill movement is automatic; steer, brake and boost are the main actions.

• **Make failure productive:** a genuine run pays for the progress already earned. Avoid long death screens or walking back uphill.

• **Make upgrades physical:** bigger wheels, spinning fans and wider frames should be visible on the vehicle.

• **Reward action:** clean landings, new checkpoints and good routes pay more than idling.

• **Keep the next run appealing:** show a reachable next purchase, a new route or a personal-best target.

What the game should feel like

An arcade toy with exaggerated physics: a fan blows the cart sideways, a spring launches it across a ravine, and a bad landing scatters cosmetic debris. Steering still needs to be predictable enough that the player knows why they failed.

Driving, controls and recovery

Action	Keyboard / mouse	Mobile
Steer	A / D or arrow keys	Large left / right controls
Brake	S or down arrow	Brake button
Boost	Space while seated	Boost button
Board / interact	E prompt	Tap prompt
Attach item	Z while valid item is selected	Weld button
Remove / reposition	Workshop UI	Workshop UI
Recover / return	On-screen Recover / Return	Same on-screen controls

Arcade physics rules

• Use gravity for downhill movement, with assisted steering and mild automatic stabilization. The stock cart must be fun before custom attachments exist.

• Use a tuned central vehicle controller for traction, braking, speed limits and steering. Do not expect arbitrary welded geometry to drive well by itself.

• Keep the camera behind the cart, widen its view gently at speed, and offer reduced shake. Avoid violent camera rolls when the cart tumbles.

• Use a forgiving health system: rough impacts reduce durability; a major crash ends the run. Show condition through clear cracks and a health bar.

• Allow one short self-right action when overturned but still intact. A stuck detector offers recovery; an early return keeps legitimately earned rewards.

Crashes that remain fun

On a crash, show a brief cart tumble and optional avatar ragdoll with no gore. The payout screen should appear in about two seconds and offer **Again** and **Garage**. Cosmetic wreckage disappears quickly and does not obstruct other players.

Friends and collisions

Begin with player-to-player cart collisions disabled during normal runs. Let players race on the same course without being blocked or shoved off by strangers. Add an explicitly selected bumper mode later if the base game supports it.

A mountain worth replaying

One course, several approaches

Build one strong mountainside before adding more worlds. It should have an easy central route, optional shortcuts and distinct landmarks. Players choose routes through driving rather than a complicated menu.

Section	Obstacle / visual identity	Reason to return
Launch ridge	Wide snow slope, forgiving ramps, first checkpoint	Learn handling and test a new part
Pine switchbacks	Wide turns with optional narrow shortcuts	Trade safety for faster times
Frozen spillway	Low grip, banked walls, coin arcs	Practice controlled slides
Broken bridge	Clear jump approach and lower rescue route	Test boost timing without an unavoidable dead end
Quarry finish	Rolling obstacles, final jump, large finish gate	Finish payout and best-time chase

Reward placement

• Place ordinary coin lines along safe paths to guide new players. Put bonus clusters on clearly visible optional jumps.

• Use broad checkpoint gates so they are easy to recognize. A checkpoint rewards first passage during a run, not repeated crossings.

• Give the first finish a clear celebration and a visible trophy in the garage. Later finishes still pay without replaying a long celebration.

• Keep moving hazards readable: show their direction, use generous gaps, and avoid unpredictable instant failures.

Variation without rebuilding the mountain

After the base course works, rotate gentle modifiers such as low gravity, extra ramps or a downhill tailwind. Announce the modifier before launch. Keep a normal route available so a temporary event does not prevent players using their builds.

Starting scope

Target roughly a minute for a competent stock-cart descent. Validate the track through repeated real driving; map length alone does not determine run time. Avoid streaming an enormous open world for the first release.

Cash, upgrades and reward pacing

One main currency

Use **Cash** for carts, normal upgrades, workshop access and attachments. Keep the launch economy understandable. Do not add several currencies to disguise the cost of progress.

Suggested payout model

Run cash = furthest-distance reward + unique pickups + capped stunt rewards + checkpoint rewards + finish bonus. Compute rewards on the server. No time-alive reward: sitting still should not be a profitable strategy.

Source	Initial tuning proposal	Abuse protection
Distance	Up to 80 cash for full course progress	Highest validated progress only
Pickups	2 cash each; about 20 accessible per run	Each pickup credited once per player per run
Stunts	5-20 per valid jump; 40 total cap	Minimum height / travel and successful landing
Checkpoints	15 each; four rewarded gates	Ordered gates; one reward each
Finish	80 cash	Valid route progress and plausible travel time

Example full run: 80 distance + 40 pickups + 30 stunts + 60 checkpoints + 80 finish = **290 cash**. An early crash earns less but keeps verified progress. These figures are a starting model, not tested balance.

Suggested purchase ladder

Purchase	Cash	Intended milestone
First wheel upgrade	100	After roughly 1-2 early runs
Basic brake / durability option	250	Try a visibly better cart
Workshop access + starter attachment kit	1,200	Target 10-15 minutes of active early play
First turbine	600 after workshop unlock	First powered custom build
Later specialist parts	1,000-4,000	New handling choices over multiple sessions

Check real earn rates before setting final prices. Every workshop unlock includes useful parts immediately; avoid making the player unlock an empty room. Use clear goals and frequent earned feedback, without punishing missed days or pressuring purchases after a crash.

The workshop: place it, press Z, go

Unlock and basic interaction

At the garage, buy Workshop Access with earned cash. The workshop includes two simple attachment parts, a placement tutorial and a free test ramp. Players can preview the feature before they can afford it.

• 1. Select an owned part from the workshop shelf or inventory. A translucent preview follows the selected mounting surface.

• 2. Position it on the cart. The preview turns green when placement is valid and red when blocked, too heavy or too far away.

• 3. Press **Z** or tap **Weld**. The part snaps to the nearest compatible mounting point with a short spark effect and click.

• 4. The server validates ownership, position, capacity and the cart state, then creates the attachment.

• 5. Select an attached item to reposition it or return it to inventory. Removing an item does not destroy a purchased part.

Keep it simple without making placement meaningless

Use generous hidden snap points on the sides, rear and top. Let players rotate parts with a visible Rotate button; Q / R can be optional shortcuts. The preview shows the direction a turbine will push. Mobile placement should not require precise dragging.

What can be welded

Owned workshop parts can attach; arbitrary scenery, other players and loose map props cannot. Only the cart owner edits the build by default. Attach and remove actions are allowed in the garage or designated safe repair pads, never during a live descent.

Persistence and safety

• Store the build as part IDs, mount IDs and allowed orientation values. Restore validated layouts when a player rejoins.

• Provide Undo Last Placement, Reset Layout and a clear capacity indicator. Reset returns parts to inventory.

• A saved layout never grants ownership of missing items. Invalid or retired parts are skipped safely with a notice.

• Never charge twice because a button was tapped repeatedly or a network response was delayed.

Important distinction: the Z/Weld input attaches a selected physical item to the cart. It does not open a generic upgrade menu. That physical assembly is a core feature.

Attachments with obvious personalities

Part	What the player notices	Tradeoff
Turbine	A spinning fan provides forward assistance	Consumes boost energy; adds weight
Rocket booster	A strong short burst on the Boost button	Long cooldown; harder cornering
Big wheels	Roll over bumps and small obstacles	Heavier and less agile
Spring suspension	Softens rough landings	Extra bounce needs steering control
Side stabilizers	Helps prevent rolling over	Uses width and mounting capacity
Glider wings	Slows falling and extends some jumps	Drag makes ground sections slower
Heavy bumper	Absorbs some frontal collision damage	More mass; less acceleration
Cosmetic horn / flag	Sound or visual personality	No competitive advantage

Capacity instead of unlimited spam

Start the custom chassis at **six capacity points**. A turbine costs two, a heavy booster three, and a light utility part one. Better chassis increase capacity modestly. Also cap the total physical part count to protect performance. Final limits depend on device testing.

Power and balance

All powered parts share one readable energy bar. Mounting four turbines does not create four independent infinite fuel supplies. Regeneration, cooldowns and a soft speed ceiling prevent runaway builds. Show combined effects before launch.

Make experimentation inexpensive

Repositioning and removing owned parts are free. Give every purchased part a short description, direction arrow and test area. Cosmetic variants can share the same gameplay statistics. Starter equipment must be able to finish the course.

Do not add at launch

Fuel crafting, wiring, manual welding accuracy, engine temperature and complex inventory weight. The fun is making a silly cart and trying it immediately, not learning engineering software.

Interface, progression and selling fairly

The screen while driving

Keep Cash, speed, condition, boost energy and checkpoint progress visible. Use short pickup feedback rather than covering the road with reward pop-ups. Workshop controls disappear during a run.

Garage layout

Put the launch pad in the center, upgrades on one side, and the locked workshop on the other. Show the next meaningful purchase and its exact cost. Friends and their carts should be visible nearby without blocking the launch lane.

Reasons to return

• **Personal bests:** furthest distance, fastest full run and cleanest landing.

• **Mastery goals:** finish without boosting, land three jumps, or reach a shortcut. Goals explain their exact requirements.

• **Visible collection:** cart bodies, paint, trails and finish trophies.

• **Build slots:** save a few layouts for a fast cart, stable cart or jump cart.

• **Friend races:** a shared countdown and finish comparison using equal-start rules.

Monetization proposal

Offer	Recommendation
Paint / trails / cart skins	Primary purchases; clear previews and prices
Garage decoration packs	Social expression without changing driving power
Extra saved build slots	Optional convenience; provide useful free slots
Private-server options	Optional custom races and modifiers
Currency / power sales	Defer until balance is proven; never require Robux to finish or access welding

Do not sell paid random rewards, use fake discounts, prompt purchases after every loss, or make a paid revive the normal way to keep a run. Review current Roblox monetization and regional requirements before implementation. No revenue outcome is guaranteed.

Accessibility

Support touch and keyboard from the first prototype. Use icons plus text, readable contrast, non-color-only placement states, optional lower effects and reduced camera shake. Make all essential actions available through buttons.

Roblox implementation plan

Suggested responsibilities

System	Responsibility
RunService / server run manager	Create run IDs, record checkpoints, validate payouts and close runs once
Vehicle controller	Seat input, assisted steering, braking, boost and stability
Attachment service	Validate placement; create WeldConstraints; apply approved part effects
Economy service	Authoritative balances, purchases and durable transaction outcomes
Player data service	Versioned cash, inventory, upgrades, layouts and settings
Client controllers	Input, camera, placement ghosts, UI and local cosmetic effects

Physics and networking

Use a central chassis and carefully configured collision groups. Attach approved parts through **WeldConstraints**; keep visual complexity low and use controlled collision geometry. Derive propulsion from registered part definitions and force direction, rather than trusting client-reported stats.

Client network ownership may improve vehicle responsiveness, but physics ownership is not proof that movement is legitimate. Validate checkpoint order, elapsed time, large position jumps and plausible speed on the server. Discard suspicious rewards without granting a client request authority over cash.

Secure attachment requests

The client sends a cart reference, owned part ID, mount ID and allowed orientation. The server checks ownership, player proximity, workshop state, available capacity and rate limits. It computes final transforms itself and rejects edits during an active run. Do not accept arbitrary instances or arbitrary force values from the client.

Data and reliability

Use Roblox DataStoreService with UpdateAsync, retries with backoff, session conflict handling and a versioned schema. Save at meaningful milestones, periodic intervals and shutdown as best effort. When player data cannot load, block purchases and progression writes rather than overwriting an existing account with defaults. Tell the player clearly.

Performance

Start with a modest server size, such as 8-12 players, as a test setting. Pool or promptly remove effects and wreckage. Measure physics, memory and frame time on lower-end mobile hardware before increasing server size or part limits.

Build order and acceptance checks

Phase	Deliverable	Exit condition
1. Driving prototype	Stock cart, graybox slope, steering, camera, reset	A full descent is controllable and fun without rewards
2. Complete run loop	Checkpoints, finish, crash payout, instant replay	A player can run, earn, upgrade and relaunch
3. Workshop prototype	Place part, Z / touch Weld, remove, test turbine	Both input types can attach and test a part without help
4. Persistent progression	Cash, inventory, builds, upgrades, error handling	Rejoining restores valid ownership and layouts
5. Content and presentation	Finished mountain, effects, tutorial, cosmetics	New players independently understand the loop
6. Limited release	Telemetry, device tests, exploit tests, balance pass	Critical failures resolved before broader promotion

Launch essentials

One mountain; one reliable stock cart; a few basic upgrades; workshop unlock; three to five tested attachments including a turbine; cash economy; saved data; mobile controls; clear crash recovery. Ship these before adding pets, rebirths, trading or several worlds.

Meaningful test cases

- Buy a part twice rapidly: it must not double-charge or grant duplicate unintended inventory.
- Weld at capacity, outside the workshop, to another cart, or with an unowned ID: reject safely.
- Use Z and the touch Weld button: both create the same validated attachment.
- Crash, disconnect, or press Return during payout: a run grants rewards no more than once.
- Rejoin with a custom build: owned parts and mount positions restore correctly.
- Simulate failed data loading: do not erase progress or accept unsafe purchases.
- Run the stock cart and strongest allowed build through every mandatory segment.
- Test multiple fully equipped carts on mobile; reject the part cap if it causes unacceptable performance.

This is a scope sequence, not a promised schedule. Estimate effort after the driving and workshop prototypes reveal physics complexity. Art production, team size and experience change delivery time substantially.

Playtest targets and first update

Measure whether the idea works

Question	What to observe	Initial target / decision
Do players start immediately?	Time from spawn to first launch	Aim for under 30 seconds
Does failure invite another go?	Crash-to-relaunch time and voluntary retries	Aim for under 10 seconds; observe frustration
Does progression move?	Runs to first upgrade; active time to workshop	About 1-2 runs; about 10-15 minutes
Is welding understandable?	First-time attachment completion without coaching	Most testers succeed; simplify if they hesitate
Is driving readable?	Repeated crashes at the same obstacle	Fix unclear geometry before adding rewards
Is it stable?	Frame time, physics faults, data/purchase errors	Resolve critical faults before scaling

These are proposed internal design targets, not measured results or Roblox benchmarks. Record real sessions before claiming they have been achieved. Review retention and purchases alongside player satisfaction rather than optimizing purchases alone.

Telemetry worth implementing

Record first launch, run start/end and reason, checkpoint reached, payout components, upgrade purchase, workshop unlock, attach attempt/result and device class. Use aggregate data; avoid collecting unnecessary personal information. Track progression bottlenecks and repeated errors.

First update after the core proves itself

- Add one new mountain route with a different driving challenge.
- Add two contrasting parts, such as a glider and heavy bumper.
- Add a short friend-race mode with a shared countdown.
- Add cosmetic trophies for clean finishes and unusual builds.

The decisions to keep

Easy to launch. Fun to crash. Worth upgrading. Cash comes from actually driving. Welding is earned with normal play and performed directly on the cart using Z or a button. The first release succeeds when players want another downhill run even before an elaborate reward system exists.

End of plan | Prepared October 24, 2026 | Working title: Cart Crash Mountain